Status	Finished
Started	Monday, 28 September 2026, 10:46 AM
Completed	Monday, 28 September 2026, 11:19 AM
Duration	32 mins 44 secs
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Given a string, determine if it is a palindrome, considering only alphanumeric characters and ignoring cases.

Note: For the purpose of this problem, we define empty string as valid palindrome.

Example 1:

```
Input:
A man, a plan, a canal: Panama

Output:
1
```

Example 2:

```
Input:
race a car

Output:
0
```

Constraints:

- `s` consists only of printable ASCII characters.

Answer: (penalty regime: 0 %)

```
import sys

def isPalindrome(s: str) -> int:
    filtered = ''.join(c.lower() for c in s if c.isalnum())
    return 1 if filtered == filtered[::-1] else 0

def main():
    # Read all lines from standard input
    lines = sys.stdin.read().splitlines()
    for line in lines:
        if line.strip():  # Skip empty lines if any
            print(isPalindrome(line))

if __name__ == '__main__':
    main()
```

	Input	Expected	Got	
✓	A man, a plan, a canal: Panama	1	1	✓
✓	race a car	0	0	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

Find if a String2 is substring of String1. If it is, return the index of the first occurrence. else return -1.

Sample Input 1

thistest123string

123

Sample Output 1

8

Answer: (penalty regime: 0 %)

```
import sys

def find_substring(s1: str, s2: str) -> int:
    return s1.find(s2)

def main():
    # Read all lines from standard input
    input_data = sys.stdin.read().splitlines()
    # Filter out empty lines just in case
    lines = [line for line in input_data if line.strip() != ""]

    if len(lines) >= 2:
        s1 = lines[0]
        s2 = lines[1]
        print(find_substring(s1, s2))

if __name__ == '__main__':
    main()
```

	Input	Expected	Got	
✓	thistest123string 123	8	8	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

Given a string s containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid.

An input string is valid if:

Open brackets must be closed by the same type of brackets.

Open brackets must be closed in the correct order.

Constraints:

1 <= s.length <= 10^4

s consists of parentheses only '()[]{}'.

For example:

Test	Result
`print(ValidParenthesis("()"))`	`true`
`print(ValidParenthesis("()[]{}"))`	`true`
`print(ValidParenthesis("(]"))`	`false`

Answer: (penalty regime: 0 %)

Reset answer

```
def ValidParenthesis(s: str) -> str:
    # An odd-length string of parentheses can never be valid
    if len(s) % 2 != 0:
        return "false"

    stack = []
    mapping = {")": "(", "}": "{", "]": "["}

    for char in s:
        if char in mapping:
            top_element = stack.pop() if stack else '#'
            if mapping[char] != top_element:
                return "false"
        else:
            stack.append(char)

    return "true" if not stack else "false"
```

	Test	Expected	Got	
✓	`print(ValidParenthesis("()"))`	`true`	`true`	✓
✓	`print(ValidParenthesis("()[]{}"))`	`true`	`true`	✓
✓	`print(ValidParenthesis("(]"))`	`false`	`false`	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Correct

Mark 1.00 out of 1.00

Assume that the given string has enough memory.

Don't use any extra space(IN-PLACE)

Sample Input 1

a2b4c6

Sample Output 1

aabbbbccccccc

Answer: (penalty regime: 0 %)

```
import sys

def expandString(s: str) -> str:
    arr = list(s)

    # 1. Parse segments from left to right to get characters and their counts
    i = 0
    segments = []
    while i < len(arr):
        char = arr[i]
        j = i + 1
        num_str = ""
        while j < len(arr) and arr[j].isdigit():
            num_str += arr[j]
            j += 1
        if num_str:
            count = int(num_str)
            segments.append((char, count))
        i = j

    # 2. Calculate total final length
    total_length = sum(count for _, count in segments)

    # Extend list with padding to simulate pre-allocated memory
    arr.extend([' '] * (total_length - len(arr)))

    # 3. Write from right to left using the pre-parsed segments
    write_ptr = total_length - 1
    for char, count in reversed(segments):
        for _ in range(count):
            arr[write_ptr] = char
            write_ptr -= 1

    return "".join(arr)

def main():
    # Read all lines from standard input
    for line in sys.stdin:
        line = line.strip()
        if line:
            print(expandString(line))

if __name__ == '__main__':
    main()
```

	Input	Expected	Got	
✓	a2b4c6	aabbbbccccccc	aabbbbccccccc	✓
✓	a12b3d4	aaaaaaaaaaaabbbdddd	aaaaaaaaaaaabbbdddd	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Correct

Mark 1.00 out of 1.00

Write a Python program to get one string and reverses a string. The input string is given as an array of characters `char[]`.

You may assume all the characters consist of printable ascii characters.

Example 1:

```
Input:
hello
Output:
olleh
```

Example 2:

```
Input:
Hannah
Output:
hannaH
```

Answer: (penalty regime: 0 %)

```
import sys

def reverseString(s: list) -> None:
    left, right = 0, len(s) - 1
    while left < right:
        s[left], s[right] = s[right], s[left]
        left += 1
        right -= 1

def main():
    # Read all lines from standard input
    for line in sys.stdin:
        line = line.strip()
        if line:
            arr = list(line)
            reverseString(arr)
            print("".join(arr))

if __name__ == '__main__':
    main()
```

	Input	Expected	Got	
✓	hello	olleh	olleh	✓
✓	Hannah	hannaH	hannaH	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

Given a **non-empty** string s and an abbreviation abbr, return whether the string matches with the given abbreviation.

A string such as "word" contains only the following valid abbreviations:

["word", "1ord", "w1rd", "wo1d", "wor1", "2rd", "w2d", "wo2", "1o1d", "1or1", "w1r1", "1o2", "2r1", "3d", "w3", "4"]

Notice that only the above abbreviations are valid abbreviations of the string "word". Any other string is not a valid abbreviation of "word".

Note:

Assume s contains only lowercase letters and abbr contains only lowercase letters and digits.

Example 1:

Input

internationalization
i12iz4n

Output

true

Explanation

Given **s** = "internationalization", **abbr** = "i12iz4n":

Return true.

Example 2:

Input

apple
a2e

Output

false

Explanation

Given **s** = "apple", **abbr** = "a2e":

Return false.

Answer: (penalty regime: 0 %)

```
import sys

def validWordAbbreviation(word: str, abbr: str) -> str:
    i = 0  # pointer for word
    j = 0  # pointer for abbr

    while i < len(word) and j < len(abbr):
        # If the abbreviation character is a digit
        if abbr[j].isdigit():
            # Leading zeros are not allowed
            if abbr[j] == '0':
                return "false"

            num = 0
            while j < len(abbr) and abbr[j].isdigit():
                num = num * 10 + int(abbr[j])
                j += 1
            # Advance word pointer by the number of skipped characters
            i += num
        else:
            # If it's a letter, it must match the word's character
            if word[i] != abbr[j]:
                return "false"
            i += 1
            j += 1

    # Both pointers must reach the end of their respective strings
    return "true" if i == len(word) and j == len(abbr) else "false"

def main():
    # Read all non-empty lines from standard input
    lines = [line.strip() for line in sys.stdin if line.strip()]

    # Process test cases in pairs (word on line 1, abbreviation on line 2)
    for k in range(0, len(lines), 2):
        if k + 1 < len(lines):
            word = lines[k]
            abbr = lines[k+1]
            print(validWordAbbreviation(word, abbr))

if __name__ == '__main__':
    main()
```

	Input	Expected	Got	
✓	internationalization i12iz4n	true	true	✓
✓	apple a2e	false	false	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

The program must accept **N** series of keystrokes as string values as the input. The character **^** represents undo action to clear the last entered keystroke. The program must print the string typed after applying the undo operations as the output. If there are no characters in the string then print **-1** as the output.

Boundary Condition(s):

1 <= N <= 100
1 <= Length of each string <= 100

Input Format:

The first line contains the integer N.
The next N lines contain a string on each line.

Output Format:

The first N lines contain the string after applying the undo operations.

Example Input/Output 1:

Input:

3
Hey ^ goooo^^glee^
lucke^y ^charr^ms
ora^^nge^^^^

Output:

Hey google
luckycharms
-1

Answer: (penalty regime: 0 %)

```
import sys

def process_keystrokes(s: str) -> str:
    stack = []
    for char in s:
        if char == '^':
            if stack:
                stack.pop()
        else:
            stack.append(char)

    result = "".join(stack)
    return result if result else "-1"

def main():
```

```
    input_data = sys.stdin.read().splitlines()
    if not input_data:
        return

    try:
        n = int(input_data[0].strip())
    except ValueError:
        n = len(input_data)

    # Process the next N lines
    count = 0
    for line in input_data[1:]:
        if count >= n:
            break
        # Note: we use line as-is (not .strip()) because spaces are valid keystrokes (e.g., "Hey ^ goooo...")
        print(process_keystrokes(line))
        count += 1

if __name__ == '__main__':
    main()
```

	Input	Expected	Got	
✓	3 Hey ^ goooo^^glee^ lucke^y ^charr^ms ora^^nge^^^^	Hey google luckycharms -1	Hey google luckycharms -1	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

Mark 1.00 out of 1.00

Given a string S which is of the format USERNAME@DOMAIN.EXTENSION, the program must print the EXTENSION, DOMAIN, USERNAME in the reverse order.

Input Format:

The first line contains S.

Output Format:

The first line contains EXTENSION.
The second line contains DOMAIN.
The third line contains USERNAME.

Boundary Condition:

1 <= Length of S <= 100

Example Input/Output 1:

Input:

abcd@gmail.com

Output:

com
gmail
abcd

For example:

Input	Result
arvijayakumar@rajalakshmi.edu.in	edu.in rajalakshmi arvijayakumar

Answer: (penalty regime: 0 %)

```
import sys

def parse_and_print(s: str):
    # Split the string by '@' into username and the rest (domain + extension)
    username, rest = s.split('@', 1)

    # Split the rest by the first dot to separate the domain from the extension
    domain, extension = rest.split('.', 1)

    # Print in reverse order: EXTENSION, DOMAIN, USERNAME
    print(extension)
    print(domain)
    print(username)

def main():
    line = sys.stdin.readline().strip()
    if line:
        parse_and_print(line)

if __name__ == '__main__':
    main()
```

	Input	Expected	Got	
✓	abcd@gmail.com	com gmail abcd	com gmail abcd	✓
✓	arvijayakumar@rajalakshmi.edu.in	edu.in rajalakshmi arvijayakumar	edu.in rajalakshmi arvijayakumar	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

A pangram is a sentence where every letter of the English alphabet appears at least once.

Given a string sentence containing only lowercase English letters, return true if sentence is a pangram, or false otherwise.

Example 1:

Input:

thequickbrownfoxjumpsoverthelazydog

Output:

true

Explanation: sentence contains at least one of every letter of the English alphabet.

Example 2:

Input:

arvijayakumar

Output: false

Constraints:

1 <= sentence.length <= 1000

sentence consists of lowercase English letters.

For example:

Test	Result
`print(checkPangram('thequickbrownfoxjumpsoverthelazydog'))`	`true`
`print(checkPangram('arvijayakumar'))`	`false`

Answer: (penalty regime: 0 %)

Reset answer

```
def checkPangram(sentence: str) -> str:
    # A pangram must contain all 26 unique lowercase English letters
    return "true" if len(set(sentence)) == 26 else "false"
```

	Test	Expected	Got	
✓	`print(checkPangram('thequickbrownfoxjumpsoverthelazydog'))`	`true`	`true`	✓
✓	`print(checkPangram('arvijayakumar'))`	`false`	`false`	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

Consider the below words as key words and check the given input is key word or not.

keywords: {break, case, continue, default, defer, else, for, func, goto, if, map, range, return, struct, type, var}

Input format:

Take string as an input from stdin.

Output format:

Print the word is key word or not.

Example Input:

break

Output:

break is a keyword

Example Input:

IF

Output:

IF is not a keyword

For example:

Input	Result
break	break is a keyword
IF	IF is not a keyword

Answer: (penalty regime: 0 %)

```
import sys

def check_keyword():
    # Define the set of keywords
    keywords = {
        "break", "case", "continue", "default", "defer", "else",
        "for", "func", "goto", "if", "map", "range", "return",
        "struct", "type", "var"
    }

    # Read the input string from standard input
    word = sys.stdin.readline().strip()

    if not word:
        return

    # Check if the exact word exists in the keyword set
    if word in keywords:
        print(f"{word} is a keyword")
    else:
        print(f"{word} is not a keyword")

if __name__ == '__main__':
    check_keyword()
```

	Input	Expected	Got	
✓	break	break is a keyword	break is a keyword	✓
✓	IF	IF is not a keyword	IF is not a keyword	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.